

Relatório de Trabalho - Fase final

José Rebelo (31516)
Professor Luís Ferreira

Barcelos

Unidade Curricular: Programação Orientada a Objetos

1 de janeiro de 2026

Conteúdo

1	Resumo	4
2	Introdução	5
3	Estado da Arte	5
3.1	Conceitos e Tecnologias	5
4	Trabalho Desenvolvido	6
4.1	Análise e Especificação	6
4.1.1	Requisitos	6
4.1.2	Arquitetura Lógica e Funcional	7
4.1.3	Modelação	7
4.1.4	Interação	7
4.2	Implementação	7
4.3	Testes e Validação	8
5	Ferramentas Utilizadas	8
5.1	Lógicas	8
5.1.1	Classes	8
5.1.2	Herança	8
5.1.3	Collections	8
5.2	Patterns	8
5.3	Softwares	9
5.3.1	Visual Studio 2026	9
5.3.2	Overleaf	9
5.3.3	Linguagem C#	9
5.3.4	GitHub	9
6	Funcionamento do código	9
6.1	Classes	9
6.2	Main	10
6.3	Validação e consistência de dados	10
7	Webgrafia	12

Lista de Figuras

1	Diagrama de Classes	11
---	-------------------------------	----

1 Resumo

O objetivo deste trabalho é desenvolver soluções em C para problemas reais de complexidade moderada, explorando conceitos de programação orientada a objetos, estruturas de dados e design de sistemas. Pretende-se identificar classes, definir estruturas de dados adequadas e implementar os principais processos que permitam suportar essas soluções, promovendo a organização e a eficiência do software desenvolvido.

Diante deste desafio, o tema escolhido por mim, entre várias opções disponibilizadas pelo professor, foi Comércio Eletrônico. O foco deste projeto consiste na criação de um sistema capaz de gerir uma loja online, contemplando funcionalidades essenciais como gestão de produtos, categorias, garantias, stocks, clientes, campanhas promocionais, vendas e marcas. Esta escolha surge da relevância crescente do comércio digital na sociedade atual, bem como do interesse em aplicar os conhecimentos de programação a um contexto prático e diretamente aplicável.

2 Introdução

Este projeto, desenvolvido no âmbito da Unidade Curricular de *Programação Orientada a Objetos* (Engenharia de Sistemas Informáticos — 1º semestre do 2º ano), tem como objetivo reforçar e aplicar os conhecimentos adquiridos ao longo do semestre, especialmente no que se refere à análise, modelação e implementação de sistemas orientados a objetos em C.

Nesta fase final do projeto, o sistema de gestão de loja online foi desenvolvido de forma significativa, implementando uma arquitetura baseada no padrão MVC (Model-View-Controller).

As classes do Model foram concluídas, incluindo a hierarquia de produtos (classe base Product e classes derivadas Shirt, FutShirt, Polo e Pants), a classe Customer para a gestão de clientes, a classe Sale para o registo de vendas e estruturas de suporte, como listas em memória, para o armazenamento de clientes, produtos e vendas.

Na View, foram desenvolvidas interfaces (ICustomerView, IProductView, ISaleView) e as respetivas implementações em consola (CustomerView, ProductView, SaleView), responsáveis pela interação com o utilizador, incluindo o pedido de dados com validação robusta, a listagem de informação e a apresentação dos resultados.

No Controller, foram criados os controladores CustomerController, ProductController e SaleController, que coordenam as regras de negócio, permitindo a adição de clientes e produtos, o registo de vendas com seleção de múltiplos itens através da referência, a listagem de registos e o cálculo de totais.

O ponto de entrada da aplicação (Program.cs) foi estruturado com a implementação de dados para teste do projeto desenvolvido onde é adicionada a inicialização das listas centrais e a correta instanciação das Views e Controllers, o que garante a separação de responsabilidades de acordo com o padrão MVC.

Este relatório está organizado de forma a demonstrar o percurso seguido durante o desenvolvimento desta fase final. Na próxima secção, são descritas as ferramentas utilizadas, tanto do ponto de vista lógico, como as listas ligadas e grafos, como também os softwares e plataformas de apoio, como o *Visual Studio*, *Overleaf*, a linguagem C, *GitHub*. Em seguida, são abordadas as principais dificuldades sentidas ao longo do processo. O documento termina com a web-grafia consultada, reunindo as fontes de apoio utilizadas no projeto.

3 Estado da Arte

3.1 Conceitos e Tecnologias

Desenvolveu-se um sistema de gestão para uma loja de roupa online, com foco na criação de classes para produtos, clientes e vendas. O projeto utiliza os princípios da Programação Orientada a Objetos, como encapsulamento, herança (para organizar os diferentes tipos de camisolas e calças) e métodos, permitindo uma estrutura clara e fácil de manter. A adoção do padrão MVC (Model-View-Controller) foi essencial para separar a lógica de negócio (Model), a interação com o utilizador (View) e o controlo das operações (Controller). Esta arquitetura torna o sistema mais organizado, facilita a correção de erros e permite adicionar novas funcionalidades no futuro, como gestão de stock ou campanhas promocionais. Para o desenvolvimento, foram usadas ferramentas como o Visual Studio para escrita e depuração do código em C#, o GitHub para controlo de versões e partilha

do projeto, e o Overleaf para a redação do relatório em L^AT_EX. Estas ferramentas ajudaram a manter o trabalho organizado, versionado e bem documentado ao longo de todo o processo. O sistema encontra-se numa fase avançada, com as principais funcionalidades implementadas (adição de clientes e produtos, registo de vendas e listagem de registos), embora ainda existam alguns erros de compilação a resolver para que a aplicação execute na totalidade. Com essas correções, o projeto ficará completo e funcional como uma aplicação de consola para gestão de loja online.

4 Trabalho Desenvolvido

4.1 Análise e Especificação

4.1.1 Requisitos

Requisitos Funcionais:

- Gestão de clientes: criação e listagem de clientes com nome, data de nascimento, NIF e telemóvel.
- Gestão de produtos: criação e listagem de diferentes tipos de produtos (camisolas de futebol, polos, calças), organizados através de herança.
- Gestão de vendas: registo de vendas associadas a um cliente, com possibilidade de adicionar múltiplos produtos e cálculo automático do total.
- Apresentação de informação: listagem de clientes, produtos e vendas, bem como detalhes individuais de cada venda.

Funcionalidades implementadas:

- Criação de classes completas com herança para os vários tipos de produtos.
- Implementação de interfaces para as Views e criação das respetivas classes de consola.
- Desenvolvimento de controladores para coordenar as operações de clientes, produtos e vendas.
- Menu principal interativo com acesso a todas as funcionalidades principais.

Requisitos Não Funcionais:

- Aplicação rigorosa de encapsulamento e boas práticas de Programação Orientada a Objetos.
- Organização do código com comentários, regiões e separação clara de responsabilidades através do padrão MVC.
- Validação básica de inputs para garantir robustez (ex: NIF e telemóvel com 9 dígitos, datas válidas).

4.1.2 Arquitetura Lógica e Funcional

O sistema foi estruturado segundo o padrão MVC (Model-View-Controller):

- **Model:** Classes de domínio (Customer, Product e derivados, Sale) e listas em memória para armazenamento temporário dos dados.
- **View:** Interfaces e classes de consola responsáveis pela interação com o utilizador (pedido de dados, validação simples e apresentação de resultados).
- **Controller:** Controladores específicos (CustomerController, ProductController, SaleController) que coordenam a lógica de negócio e a comunicação entre Model e View.

Esta arquitetura promoveu uma separação clara de responsabilidades, facilitou o desenvolvimento incremental e permitiu testes isolados por componente.

4.1.3 Modelação

Modelação de Dados: Os produtos são representados por uma hierarquia de classes (Product → Shirt → FutShirt/Polo; Product → Pants), com atributos comuns (preço, fabricante, referência) e específicos de cada tipo. Os clientes possuem nome, data de nascimento, NIF e telemóvel. As vendas associam um NIF de cliente, data e lista de produtos, com método para cálculo do total.**Integração:** A integração foi realizada de forma progressiva, começando pela criação independente das classes do Model, seguida da implementação das Views e Controllers, culminando na ligação através do menu principal no Program.cs.

4.1.4 Interação

O sistema oferece as seguintes funcionalidades principais através de um menu em consola:

- Criação e listagem de clientes;
- Criação e listagem de produtos de diferentes tipos;
- Registo de vendas com seleção de cliente por NIF e adição de múltiplos produtos por referência;
- Listagem de vendas.

4.2 Implementação

Foram implementadas as seguintes operações principais:

- Inserção de clientes com validação de NIF e telemóvel;
- Inserção de produtos com suporte a vários tipos através de menu e herança;
- Registo completo de vendas com cálculo automático do valor total;
- Listagem estruturada de clientes, produtos e vendas.

O desenvolvimento seguiu uma abordagem iterativa: cada componente (Model, View, Controller) foi implementado e testado separadamente antes de ser integrado no sistema global.

4.3 Testes e Validação

Para garantir a qualidade do sistema, foram realizados testes manuais extensivos em cada etapa:

- **Testes de Criação:** Verificação da correta instanciação de clientes e produtos de todos os tipos, com preenchimento adequado dos atributos específicos.
- **Testes de Vendas:** Confirmação do registo de vendas com múltiplos produtos, associação correta ao cliente e cálculo preciso do total.
- **Testes de Listagem:** Validação da apresentação clara e completa de clientes, produtos e vendas.

5 Ferramentas Utilizadas

Durante a realização do projeto foram utilizadas diversas ferramentas que desempenharam um papel crucial em cada etapa do trabalho. Cada ferramenta tem um propósito diferente.

5.1 Lógicas

5.1.1 Classes

As classes são a base da programação orientada a objetos e permitem definir tipos de objetos com atributos e métodos próprios. Nesta fase do projeto, foram criadas classes para representar produtos e clientes. Cada classe encapsula os dados relevantes, como nome, preço, etc., e métodos para gerir, adicionar itens, garantindo que a manipulação de informação seja consistente e organizada.

5.1.2 Herança

A herança permite que uma classe "filha" reutilize atributos e métodos de uma classe "pai", que ajuda na redução de código duplicado. No projeto, os diferentes tipos de camisolas foram modelados com herança: uma classe base define os atributos e métodos comuns a todas as camisolas, enquanto subclasses específicas adicionam características próprias, como a cor ou estilo. Isto facilita a extensão do sistema sem alterar as classes já existentes.

5.1.3 Collections

As coleções (*collections*) permitem armazenar e gerir conjuntos de objetos de forma dinâmica. Neste projeto, foram utilizadas listas (`List<T>`) para guardar clientes, produtos e vendas, facilitando operações como adição, pesquisa e listagem. A utilização de collections contribui para uma organização clara e eficiente da informação ao longo da aplicação.

5.2 Patterns

Os design *patterns* são soluções reutilizáveis para problemas comuns de programação. No projeto, foram aplicados padrões como MVC para separar claramente dados, lógica de negócio e apresentação, garantindo organização, manutenção facilitada e código mais estruturado.

5.3 Softwares

5.3.1 Visual Studio 2026

O *Visual Studio* é um ambiente de desenvolvimento integrado (IDE) desenvolvido pela *Microsoft*. Ele oferece um conjunto abrangente de ferramentas para criar aplicações para diversas plataformas, incluindo *Windows*, *web*, dispositivos móveis e jogos. Com suporte para várias linguagens de programação, como *C#*, *C++*, *Python* e *JavaScript*. Recorri ao uso do *Visual Studio* porque é uma ferramenta no qual estou confortável a usar.

5.3.2 Overleaf

O *Overleaf* é uma plataforma online destinada à criação, edição e colaboração em documentos *LaTeX*. Permite que múltiplos utilizadores possam editar simultaneamente o mesmo documento, com as alterações sendo refletidas em tempo real. Além disso, preconiza a compilação automática e a integração com serviços como o *GitHub*, tornando a elaboração de documentos técnicos e académicos muito mais rápida e prática. Recorri ao Overleaf para a criação do relatório em *LaTeX*.

5.3.3 Linguagem C#

A linguagem *C#* (*C Sharp*) é uma linguagem moderna, orientada a objetos, concebida para oferecer um desenvolvimento mais seguro, organizado e eficiente. Combina uma sintaxe clara com um forte controlo de tipos, evitando erros comuns associados à gestão manual de memória. Para além disso, inclui uma vasta biblioteca padrão e suporte para programação modular, o que facilita a criação, manutenção e expansão de aplicações de média e grande escala. No âmbito deste projeto, foi utilizada esta linguagem devido à clareza e adequação ao desenvolvimento de sistemas estruturados baseados em classes e hierarquias, foi também solicitado pelo professor o seu uso.

5.3.4 GitHub

O *GitHub* é uma plataforma de hospedagem de código para controlo da versão e colaboração. Ele permite que desenvolvedores trabalhem juntos em projetos, rastreiem alterações e partilhem código. Recorri ao uso do *GitHub* para armazenar o meu projeto.

6 Funcionamento do código

6.1 Classes

O projeto está organizado segundo o paradigma da programação orientada a objetos, com classes que representam os diferentes elementos do sistema. As classes podem ser divididas em *Models*, que contêm os dados e regras de negócio, e classes de *herança*, que representam produtos específicos.

Entre as classes de produtos, temos a classe **Shirt** como classe base para camisas, da qual derivam **FutShirt** (camisas de futebol) e **Polo**. A classe **Pants** representa calças, derivando da classe base **Product**, que é comum a todos os produtos da loja. Estas classes encapsulam atributos relevantes, como preço, fabricante, referência, tamanho, cor, tipo de manga ou equipa, e métodos para aceder ou alterar estes dados de forma consistente.

As classes `Customer` e `Sale` permitem gerir clientes e registar vendas, respetivamente, enquanto a classe `Expencc` é responsável por calcular o total gasto por cada cliente, agregando as vendas associadas.

O projeto também inclui *controllers*, como `CustomerController`, `ProductController`, `SaleController` e `ExpenccController`, que coordenam a lógica de negócio: adicionam clientes e produtos, registam vendas com múltiplos itens, calculam totais e interagem com as views. Por outro lado, as *views* (`CustomerView`, `ProductView`, `SaleView`) implementam interfaces (`ICustomerView`, `IProductView`, `ISaleView`) para interagir com o utilizador através da consola, pedindo dados, listando informações e mostrando resultados de forma clara.

Esta estrutura segue o padrão *MVC* (*Model-View-Controller*), garantindo separação de responsabilidades:

- **Model:** classes que armazenam dados e regras de negócio (`Customer`, `Product`, `Sale`, `Expencc`).
- **View:** interfaces e implementações que tratam a interação com o utilizador (`CustomerView`, `ProductView`, `SaleView`).
- **Controller:** classes que gerem a lógica e coordenam Models e Views (`CustomerController`, `ProductController`, `SaleController`, `ExpenccController`).

6.2 Main

O `Main` é o ponto de entrada da aplicação, responsável por iniciar o sistema e organizar a execução da lógica construída. Neste ponto, são inicializadas as listas centrais (`List<Customer>`, `List<Product>`, `List<Sale>`), assim como as instâncias das views e controllers.

A partir do `Main`, é possível:

- Criar novos clientes e produtos através dos controllers correspondentes, utilizando métodos que pedem informações ao utilizador e validam os inputs.
- Registar vendas, permitindo a seleção de múltiplos produtos por referência, associando-os a um cliente específico.
- Listar clientes, produtos e vendas, exibindo informações detalhadas de forma organizada.
- Calcular o total gasto por cada cliente, utilizando a classe `Expencc` e apresentando o resultado pela view correspondente.

Assim, o `Main` funciona como centro de coordenação de todo o sistema, integrando classes, controllers e views, garantindo que cada funcionalidade é executada corretamente e na ordem adequada, enquanto mantém a separação clara de responsabilidades definida pelo padrão MVC.

6.3 Validação e consistência de dados

O sistema inclui mecanismos de validação para assegurar que os dados introduzidos pelo utilizador são consistentes:

- NIF e telemóvel com 9 dígitos.
- Datas de nascimento válidas.
- Referências de produtos existentes na lista de produtos.
- Tipos e atributos de produtos corretos (tamanho, cor, tipo de manga, equipa, classe da camisola, etc.).

Estes mecanismos garantem que a aplicação funciona de forma estável, robusta e intuitiva, mesmo em ambiente de consola.

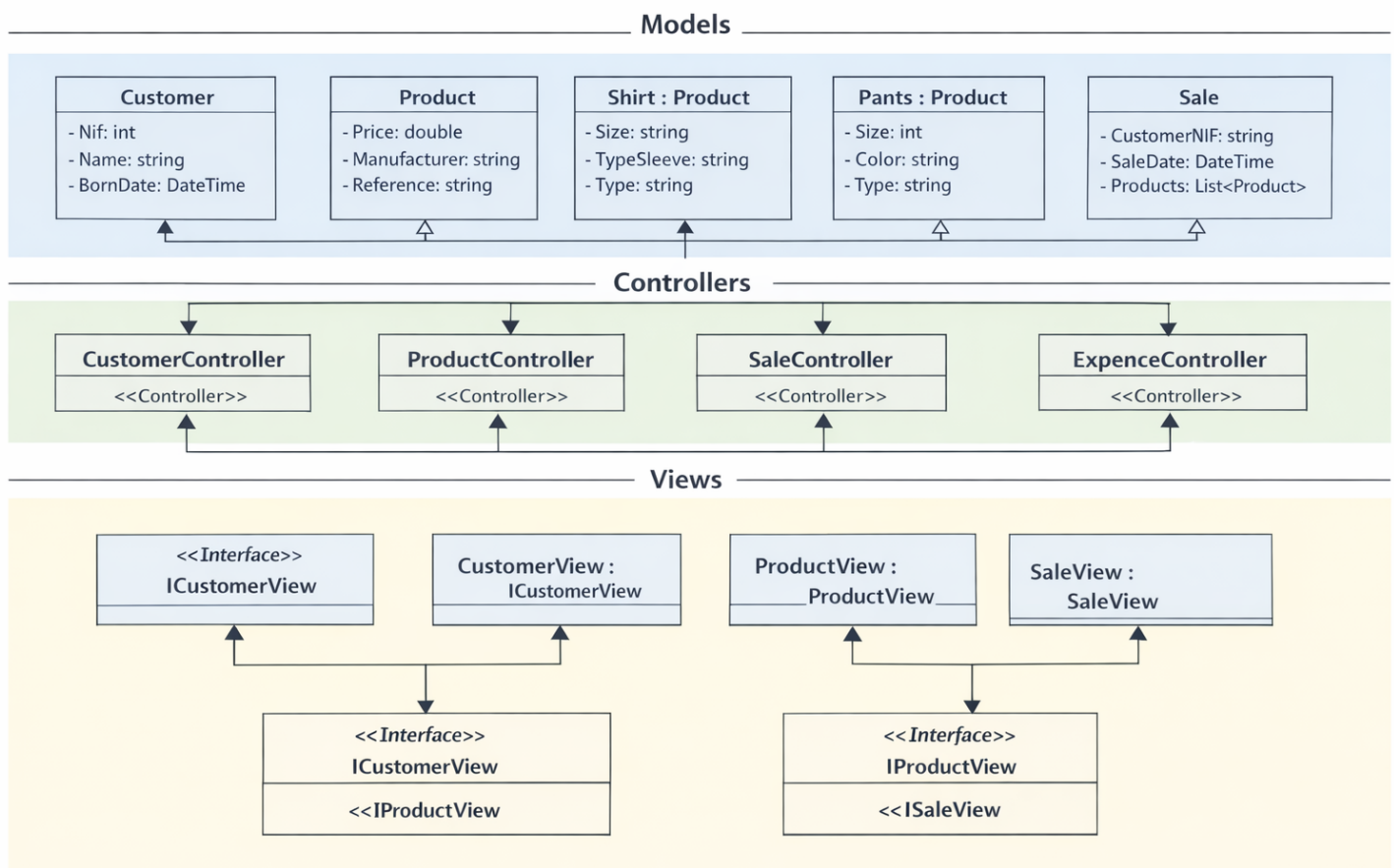


Figura 1: Diagrama de Classes

7 Webgrafia

Referências

[1] Repositório GitHub. Visitar repositório